

A Project Report on

Negotiating Multi Agent Red/Blue Team Simulator

for the course CIP81 - PROJECT WORK

Submitted in partial fulfillment of the requirements for the award of the degree of

**Bachelor of Engineering in Computer Science and Engineering
(Artificial Intelligence & Machine Learning)**

By:

Sargam Nema 1MS22CI057

Shobhit Srivastava 1MS22CI063

Aryaman Prakash 1MS22CY010

Suryansh Prajapati 1MS22CY069

Under the Guidance of:

Dr. Nithya N

Associate Proffessor

M S RAMAIAH INSTITUTE OF TECHNOLOGY

(Autonomous Institute, Affiliated to VTU)

BANGALORE-560054

www.msrit.edu

2025-2026

CERTIFICATE

Certified that the Project Work(CIP81) entitled “Negotiating Multi Agent Red/Blue Team Simulator” carried out by Sargam Nema – 1MS22CI057, Shobhit Srivastava – 1MS22CI063, Aryaman Prakash – 1MS22CY010, Suryansh Prajapati – 1MS22CY069 bonafide students of M. S. Ramaiah Institute of Technology Bengaluru in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering (AI&ML) of the Visvesvaraya Technological University, Belagavi during the year 2025-26. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the department. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said degree.

Signature of the Guide

Dr. Nithya N

Associate Professor

Dept. of CSE(Cyber Security)

Signature of the HoD

Dr. Siddesh G M

Professor & Head

Dept. of CSE(AI&ML)&CSE(Cyber Security)

External Examiners

Name of the Examiners:

1.

2.

Signature with Date

DECLARATION

We, hereby, declare that the entire work embodied in this project report has been carried out by us at M. S. Ramaiah Institute of Technology, Bengaluru, under the supervision of Faculty Name, Designation, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning). This report has not been submitted in part or full for the award of any diploma or degree of this or to any other university.

Signature
Sargam Nema
1MS22CI057

Signature
Shobhit Srivastava
1MS22CI063

Signature
Aryaman Prakash
1MS22CY010

Signature
Suryansh Prajapati
1MS22CY069

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project. We would like to express our profound gratitude to the Management and **Dr. N.V.R Naidu** Principal, M.S.R.I.T, Bengaluru for providing us with the opportunity to explore our potential.

We extend our heartfelt gratitude to our beloved **Dr. Siddesh G M**, HOD, Department of Computer Science and Engineering (AI&ML), for constant support and guidance.

We wholeheartedly thank our project guide **Nithya N**, for providing us with the confidence and strength to overcome every obstacle at each step of the project and inspiring us to the best of our potential. We also thank **Dr. Sini Anna Alex**, Project Coordinator for her constant guidance, direction and insight during the project.

This work would not have been possible without the guidance and help of several individuals who in one way or another contributed their valuable assistance in preparation and completion of this study.

Finally, we would like to express sincere gratitude to all the teaching and non-teaching faculty of Computer Science and Engineering (AI&ML) Department, our beloved parents, seniors and my dear friends for their constant support during the course of work.

Sargam Nema 1MS22CI057

Shobhit Srivastava 1MS22CI063

Aryaman Prakash 1MS22CY010

Suryansh Prajapati 1MS22CY069

Abstract

This Project (Sandboxed Red vs Blue Autonomous Cyber Defense Simulation System) is a research-oriented platform developed to support safe, explainable, and reproducible experimentation in autonomous cyber defense. The project models attacker-defender interactions within a synthetic graph-based network, enabling controlled evaluation of red-team and blue-team strategies without relying on real exploit frameworks, live targets, or unsafe operational tooling. Its architecture emphasizes modularity, allowing independent development of environment, agent, simulation, observability, metrics, and experiment components.

The system is designed to address key research needs in cybersecurity simulation, including deterministic execution, action-level explainability, and experiment repeatability through seed-based runs and provenance tracking. This Project includes a completed network simulation core, deterministic rule-based red and blue agents, adaptive planning-based autonomy, observability and visualization pipelines, comparative reporting utilities, and advanced research extensions such as deception hooks, scenario taxonomy support, stress testing, and ablation workflows. By combining sandboxed execution with structured logging, policy evaluation, and reproducible experimentation, This Project provides a practical foundation for studying autonomous cyber defense strategies and benchmarking intelligent agent behavior in a secure simulation setting.

Contents

Acknowledgement	i
Abstract	ii
List of Figures	v
List of Tables	vi
1 INTRODUCTION	1
1.1 General Introduction	1
1.2 Problem Statement	1
1.3 Objectives of the project	2
1.4 Project deliverables	2
1.5 Current Scope	3
1.6 Future Scope	4
2 LITERATURE SURVEY	5
2.1 Introduction	5
2.2 Related Works	6
2.3 Conclusion of Survey	8
3 PROJECT REQUIREMENT SPECIFICATIONS	9
3.1 Hardware Requirements	9
3.2 Software Requirements	10
3.3 Functional requirements	10
3.4 Non-Functional requirements	11
4 DESIGN	13
4.1 Introduction	13

4.2	Architecture Design	13
4.3	User Interface Design	14
4.4	Low Level Design	24
5	IMPLEMENTATION	27
5.1	Pseudocode	27
5.2	Explanation	30
6	RESULTS	33
6.1	Result Snapshots	33
6.2	Comparison	35
6.3	Performance Analysis	36
7	CONCLUSION & FUTURE WORK	40
7.1	Conclusion	40
7.2	Future work	41
	References	42

List of Figures

4.1	Architecture Diagram	14
4.2	Package Level Architecture Diagram	15
4.3	Runtime execution Flow	16
4.4	Simulation Sequence	17
4.5	Observability and Artifact pipeline	17
4.6	Experimnet and Reporting Workflow	18
4.7	Deployment and Access Surface	19
4.8	User Interaction Overview	19
4.9	CLI Navigation Flow	20
4.10	Notebook-Based Analysis UI	20
4.11	Core Module Interaction	21
4.12	Timestamp Execution Flow	22
4.13	Policy Decision Internals	23
4.14	Artifact Generation Pipeline	24
4.15	Scenario Validation and Graph Build	25
6.1	Result snapshot of the Rule Red vs Rule Blue baseline execution.	33
6.2	Result snapshot of the Adaptive Red vs Rule Blue execution.	34
6.3	Snapshot of the comparative report generated between the baseline run and the adaptive Red run.	35
6.4	Compromise trend comparison between the baseline run and the Adaptive Red vs Rule Blue run.	37
6.5	Defense efficiency comparison using containment-to-compromise ratio.	37
6.6	Response latency comparison between the baseline run and the adaptive Red run. . .	38

List of Tables

6.1	Multi-seed baseline summary for the rule-baseline scenario.	35
6.2	Comparison of selected policy combinations from the policy experiment matrix. . . .	36
6.3	Stress-test profile summary generated from the stress-test suite.	38

Chapter 1

INTRODUCTION

1.1 General Introduction

Cybersecurity defense research increasingly requires safe environments where attack and defense strategies can be studied without exposing real systems to risk. In real-world settings, testing autonomous attack and defense behavior is costly, unsafe, and difficult to reproduce. This Project addresses this need by providing a sandboxed Red vs Blue simulation platform in which attacker and defender agents interact inside a synthetic graph-based network.

The project is designed as a research-grade system rather than just a simple simulator. It emphasizes modularity, explainability, and reproducibility. The platform models network nodes, vulnerabilities, services, defensive controls, agent actions, and state transitions in a controlled environment. It supports both deterministic rule-based agents and adaptive planning-based agents, while also generating structured logs, comparative reports, experiment outputs, and visualizations. Because the entire system is abstract and sandboxed, it can be safely used for academic experimentation, algorithm evaluation, and autonomous cyber defense studies.

1.2 Problem Statement

Existing cybersecurity experimentation approaches often suffer from one or more major limitations: they are not safe for repeated testing, they lack clear explainability of agent decisions, they are difficult to reproduce across runs, or they do not provide standardized artifacts for research comparison. Many ad hoc simulators also mix environment logic, agent behavior, and evaluation logic too tightly, making them hard to extend or validate.

The problem this project addresses is the lack of a controlled, modular, and reproducible platform

for studying attacker-defender dynamics in autonomous cyber defense. A system is needed where Red and Blue strategies can be evaluated under identical conditions, where every action can be justified and traced, and where experiment results can be repeated, compared, and analyzed with confidence.

1.3 Objectives of the project

The main objective of This Project is to design and implement a sandboxed autonomous cyber defense simulation system for Red vs Blue experimentation.

Specific objectives are:

- To model a synthetic network environment using graph-based structures.
- To simulate attacker and defender interactions through abstract legal actions.
- To implement deterministic rule-based Red and Blue baseline agents.
- To support adaptive planning-based autonomous policies for comparative evaluation.
- To ensure explainability by attaching rationale and expected effects to agent actions.
- To ensure reproducibility through seeded execution, provenance capture, and structured artifacts.
- To provide observability, metrics, replay, visualization, and comparative reporting capabilities.
- To support research workflows such as multi-seed testing, stress experiments, scenario variation, and ablation studies.
- To package the system with CLI commands, configuration presets, notebooks, and container support for easier use and delivery.

1.4 Project deliverables

The current repository indicates the following major deliverables:

- A graph-based network simulation core with state modeling, transitions, legal action generation, and seeded execution.
- Schema-driven scenario validation and a reusable scenario catalog.
- Rule-based Red and Blue policy implementations.

- Adaptive planning and deception-aware policy components.
- A turn-based simulation orchestrator with structured logging and state-diff tracking.
- Observability and provenance modules for experiment traceability.
- Metrics and baseline evaluation modules.
- Experiment runners for multi-seed reports, policy matrices, stress-test suites, and ablation reports.
- Visualization and comparative report generation, including markdown and SVG outputs.
- Config presets and seed bundles for repeatable experiment execution.
- Packaged CLI entry points for running experiments and validation workflows.
- Docker assets for containerized execution.
- Notebook-based analysis workflows for replay, comparative study, and ablation interpretation.
- Automated test suites covering unit, integration, simulation, reproducibility, and regression behavior.
- Release-readiness validation support for checking packaging, scripts, configs, notebooks, and delivery assets.

1.5 Current Scope

At present, the scope of This Project is focused on safe, research-oriented cyber defense simulation in a fully sandboxed environment. The project currently supports synthetic network modeling, abstract offensive and defensive actions, deterministic baseline agents, adaptive planning policies, experiment orchestration, logging, reporting, visualization, and reproducibility validation.

The system does not operate on live infrastructure and does not include real exploit frameworks, real offensive tooling, or external network interaction. Its current scope is limited to controlled simulation, experiment management, evaluation, and research artifact generation. Productization is partially complete: packaging, CLI support, Docker assets, notebooks, and release-readiness checks are present, while broader collaborator setup and final delivery polish are still being expanded.

1.6 Future Scope

The project has strong potential for future extension in both research and engineering directions.

Possible future scope includes:

Integration of more advanced adaptive agents using LLM-based reasoning or reinforcement learning methods. Expansion of the scenario library to include larger, more realistic, and more diverse enterprise-scale network topologies. Improved observability models such as partial visibility, uncertain sensing, and richer deception behavior. More sophisticated scoring, utility shaping, and multi-objective optimization for Red and Blue policies. Enhanced visualization dashboards for deeper replay and trajectory analysis. Additional benchmark suites for robustness, adversarial adaptation, and resilience under changing conditions. Web-based or hosted interfaces for easier experiment management and collaboration. Stronger productization through simplified installer flows, richer documentation, and end-user deployment support. Possible educational use as a safe academic platform for cybersecurity training and autonomous defense experimentation.

Chapter 2

LITERATURE SURVEY

2.1 Introduction

Cybersecurity research has increasingly moved toward automated and intelligent defense because modern attacks are fast, adaptive, and difficult to analyze using only manual or static methods. In particular, researchers have explored automated adversary emulation, cyber deception using honeypots, reinforcement learning for adaptive defense, and multi-agent cyber battle simulation. These directions are important because they help defenders study attacker behavior in controlled environments, evaluate defense strategies, and improve resilience without exposing real systems to unnecessary risk.

One major area of research is automated adversary emulation, where attacker behaviors are represented and executed in a structured way to assess an organization's defenses. Another important area is cyber deception, especially honeypots and adaptive honeypots, which are designed to lure attackers, delay them, and collect useful threat intelligence. More recent work has focused on reinforcement learning and multi-agent learning, where defensive agents can improve their strategies through repeated interaction with simulated attack scenarios. These studies show that adaptive and autonomous defense is a promising direction, but they also reveal practical limitations related to realism, explainability, safety, and reproducibility.

The present project, MARBTS, is motivated by these research trends. The literature suggests that while many systems exist for adversary emulation, deception, or adaptive cyber defense, fewer platforms combine all of these ideas within a single safe, modular, explainable, and reproducible research environment. Therefore, a sandboxed Red vs Blue simulation platform is needed to support controlled experimentation and systematic comparison of attacker and defender policies.

2.2 Related Works

Miller et al. introduced an important conceptual foundation for automated adversary emulation through their MITRE work on planning and acting under uncertainty [1]. They argued that adversary emulation should not be treated as a pure planning problem alone, because real attackers operate in uncertain environments and must continuously adapt during execution. Their work formalized the separation between planning and acting, and this idea became highly influential in later adversary emulation systems. The main contribution of this study is that it framed automated cyber offense as a dynamic decision process rather than as a fixed script. However, the work mainly focused on adversary emulation logic and not on broader explainable Red-Blue research workflows. This is relevant to the current project because MARBTS also separates policy reasoning from action execution, but it does so inside a sandboxed and research-oriented simulation platform.

MITRE CALDERA extended this idea into a practical open-source platform for automated adversary emulation [2]. CALDERA uses the ATT&CK framework to organize adversary behaviors and allows automated security assessments based on post-compromise techniques. Its importance lies in demonstrating that automated adversary emulation is not merely theoretical, but can be implemented as a real operational system used by cybersecurity practitioners. However, CALDERA is primarily designed for live or enterprise-style emulation rather than for controlled academic experimentation between explainable Red and Blue agents. This limitation makes it highly relevant to MARBTS, which aims to provide a safer and more reproducible simulation-based alternative for experimentation.

Priya and Chakkaravarthy studied containerized cloud-based honeypot deception for tracking attackers [3]. Their work showed that containerized honeypot systems are portable, scalable, and practical for collecting telemetry about attacker behavior. A key contribution of this study is that it demonstrates how cloud and container technologies can support safe and manageable deception deployments. However, the work is focused more on honeypot deployment and telemetry collection than on autonomous adaptation or Red-Blue decision-making. For the current project, this study is relevant because it supports the use of safe, isolated, and instrumented environments for cyber defense experimentation.

Wang, Li, and Chen explored the use of reinforcement learning for Blue Team defense against ransomware in simulated environments [4]. Their objective was to improve detection, response, and mitigation performance using an RL-based defender. The study reported better response speed, higher response accuracy, and stronger resilience than traditional methods. This work is directly relevant because it demonstrates the value of adaptive defensive policies in simulated cyber environments.

At the same time, it is limited to a ransomware-focused simulation and does not provide a general modular Red-Blue framework. This creates a clear connection to MARBTS, which is broader in scope and supports comparative experimentation across multiple conditions.

Javadpour et al. presented a comprehensive survey on cyber deception techniques for improving honeypot performance [5]. Their work categorized deception methods, compared their strengths and limitations, and identified research gaps in honeypot realism, detectability, and evaluation. One important contribution of this survey is that it provides a taxonomy of deception mechanisms and discusses the conditions under which deception remains effective against adaptive attackers. Since MARBTS includes deception-aware extensions and adaptive-policy hooks, this survey is valuable for grounding the project's deception concepts in established research. Its limitation is that it is a survey and does not itself provide an experimental Red-Blue simulation framework.

Morozov et al. proposed an adaptive honeypot framework for IoT environments [6]. Their work emphasized dynamically adjusting honeypot behavior based on observed attacks and introduced an evaluation-oriented framework for IoT deception. The paper showed the value of adaptive deception in improving engagement and realism. Although its focus is specifically on IoT systems, the broader contribution is the idea that deception systems should evolve in response to adversary behavior. This idea is relevant to MARBTS because the project also explores deception primitives and adaptive policies, though in a more general abstract network setting rather than only IoT networks.

Landolt et al. surveyed the application of multi-agent reinforcement learning in cybersecurity [7]. Their study highlighted the potential of MARL for decentralized, adaptive, and collaborative cyber defense. They also pointed out major research challenges such as scalability, non-stationarity, adversarial robustness, and the need for realistic training environments. This survey is especially important for the current project because it justifies the use of simulation as a training and evaluation space for adaptive agents. It also supports the idea that future cyber defense systems may need interacting Red and Blue agents rather than only single-agent optimization.

Xie et al. proposed the Nash Q-Network for multi-agent cybersecurity simulation [8]. Their work addressed the instability of simultaneous multi-agent learning by introducing a Nash-oriented approach for two-player zero-sum cyber environments. The study demonstrated that equilibrium-based learning can support more stable and robust cyber defense strategies against adaptive attackers. This paper is highly relevant to MARBTS because it suggests a concrete future direction for learning attacker and defender policies in adversarial settings. However, like many MARL studies, it remains focused on algorithmic performance inside a specific learning environment rather than on explainability and experiment packaging.

Soulé et al. presented a multi-agent simulation framework for cyber-attackers and cyber-defenders

based on Markovian modeling [9]. Their work abstracted nodes as sets of properties and modeled actions as state-changing operations that move the environment toward attacker or defender goals. This is one of the most closely related works to MARBTS because it explicitly studies cyber conflict through interacting attacker and defender agents in simulation. Its major contribution is the use of a structured state-transition framework for coordinated attack and defense experiments. However, the work does not place as much emphasis on reproducibility workflows, artifact generation, or action-level explainability as the current project does.

2.3 Conclusion of Survey

The literature survey shows that prior research has made important progress in four connected areas: automated adversary emulation, cyber deception, reinforcement learning for adaptive defense, and multi-agent cyber simulation. MITRE's work established the importance of planning and acting under uncertainty in adversary emulation [1,2]. Honeypot and deception research demonstrated the value of safe, observable, and adaptive defensive traps [3,5,6]. Reinforcement learning and multi-agent studies showed that adaptive defenders can outperform static strategies under certain simulated conditions [4, 7, 8]. Finally, simulation-oriented research confirmed that attacker-defender interaction can be represented in a formal and controlled environment for experimentation [9].

At the same time, the survey reveals a clear gap. Existing works often focus on only one dimension of the problem. Some emphasize operational adversary emulation, some focus on honeypot deployment and telemetry collection, while others study adaptive learning algorithms in specialized simulations. Few systems combine modular Red-Blue interaction, explainability, experiment reproducibility, structured artifact generation, and safety through abstraction within a single platform.

This gap leads directly to the motivation for the current study. MARBTS is being developed because there is a need for a sandboxed, research-grade platform where Red and Blue strategies can be modeled, compared, explained, and reproduced safely. In this sense, the present project builds on the strengths of prior work while addressing their limitations through a unified simulation framework for autonomous cyber defense experimentation.

Chapter 3

PROJECT REQUIREMENT SPECIFICATIONS

3.1 Hardware Requirements

Since this Project is a Python-based sandboxed cyber defense and attack simulation platform, it does not require specialized hardware such as GPUs, dedicated servers, or cybersecurity lab equipment. It can run on a standard development machine.

Recommended hardware requirements are:

- Processor: Intel Core i5 / AMD Ryzen 5 or above
- RAM: Minimum 8 GB, recommended 16 GB for running larger experiment batches
- Storage: Minimum 2 GB free disk space for source code, dependencies, generated artifacts, reports, and figures
- Display: Standard monitor for development, visualization, and notebook-based analysis
- Network: Internet connection only for dependency installation or repository access; not required for simulation execution itself.

The project is designed so that all simulations occur in a synthetic environment. Therefore, no live target hardware, network appliances, or penetration-testing devices are required.

3.2 Software Requirements

- Operating System: Linux, Windows, or macOS
- Python Version: Python 3.11 or above
- Container Runtime: Docker and Docker Compose for containerized execution
- Package/Build Tools: pip, setuptools, wheel
- Testing Framework: pytest
- Core Python Libraries: networkx for graph-based network modeling, matplotlib for visualization and report figures, packaging for versioning and runtime support
- Development Environment: Any code editor or IDE such as VS Code or PyCharm Command-line shell such as Bash, Zsh, or PowerShell

3.3 Functional requirements

The main functional requirements of this Project, based on the implemented project structure, are as follows:

- The system shall model a synthetic computer network as a graph of interconnected nodes and edges.
- The system shall support scenario loading and schema validation for predefined baseline and library scenarios.
- The system shall maintain environment state including node attributes, compromise state, detection state, and isolation state.
- The system shall generate and enforce legal Red and Blue actions according to environment constraints.
- The system shall support Red team actions such as scanning, exploitation, lateral movement, and privilege escalation in abstract form.
- The system shall support Blue team actions such as monitoring, patching, blocking, and isolating in abstract form.

- The system shall provide a turn-based simulation kernel to execute attacker-defender interactions over time.
- The system shall support deterministic seeded execution for reproducible results.
- The system shall implement rule-based Red and Blue baseline policies.
- The system shall support adaptive planning-based policy components and deception-aware hooks for research experimentation.
- The system shall record action rationale, expected effect, and policy traces for explainability.
- The system shall generate structured logs, state differences, provenance metadata, and run artifacts for each simulation.
- The system shall compute baseline metrics and comparative experiment outcomes.
- The system shall support multi-seed experiment execution and aggregated reporting.
- The system shall support policy experiment matrices, stress-test suites, and ablation report generation.
- The system shall provide replay, comparative report packaging, and visualization outputs such as summaries and figures.
- The system shall support configuration presets and curated seed bundles for repeatable workflow execution.
- The system shall provide CLI and script-based interfaces for experiment execution.
- The system shall validate release readiness through automated checks on packaging, configs, scripts, notebooks, and tests.

3.4 Non-Functional requirements

The non-functional requirements are especially important in this project because it is intended for research and controlled experimentation rather than operational deployment.

- **Safety:** The system must remain fully sandboxed and abstract, with no real exploit code, no unsafe offensive tooling, and no live target interaction.

- **Reproducibility:** The platform must support repeatable seeded runs and capture provenance details so that the same experiment can be reproduced consistently.
- **Explainability:** Every agent action should include rationale, predicted effect, and traceable decision context to support analysis and debugging.
- **Modularity:** Core components such as environment, agents, simulation, logging, metrics, and experiments must remain replaceable through stable interfaces.
- **Maintainability:** The repository should preserve clear separation of concerns, organized structure, and contract-based shared models.
- **Scalability:** The architecture should support expansion to larger scenario sets, experiment matrices, and stress-test workflows.
- **Testability:** The system must provide unit, integration, simulation, reproducibility, and regression testing support.
- **Portability:** The project should run across common operating systems and support both local and containerized execution.
- **Usability:** Canonical scripts, CLI commands, config presets, and notebooks should make the platform easier for researchers and collaborators to use.
- **Reliability:** Validation checks, schema enforcement, and release-readiness gates should reduce configuration errors and incomplete deliveries.
- **Research Utility:** The platform should produce structured artifacts, metrics, reports, and visualizations suitable for academic experimentation and comparative analysis.

Chapter 4

DESIGN

4.1 Introduction

This Project is designed as a modular, research-oriented cyber defense simulation platform for studying Red vs Blue behavior in a safe and fully sandboxed environment. The design is centered on a graph-based synthetic network, interchangeable attacker and defender policies, a turn-based simulation kernel, and a strong observability pipeline for logging, replay, and reporting. Rather than combining all logic in one monolithic program, the project separates shared contracts, environment logic, agent behavior, simulation control, experiment execution, and report generation into distinct modules.

A key design characteristic of This Project is that it treats explainability and reproducibility as first-class design goals. Every agent action is represented as a legal action object, selected through a policy layer, applied through controlled transition functions, and then written into structured artifacts together with rationale, metrics, state differences, and provenance metadata. This makes the system suitable for research experiments, baseline comparison, and extension toward more advanced adaptive autonomy.

4.2 Architecture Design

The implemented architecture follows a layered design.

The shared foundation layer is `src/hart/`, which contains the canonical enums and dataclass-based models used across the project. This prevents contract duplication and keeps all major subsystems aligned to the same vocabulary and data structures. The input and schema layer is formed by `scenarios/`, `configs/`, and `src/schemas/`. Scenario JSON files are validated and converted

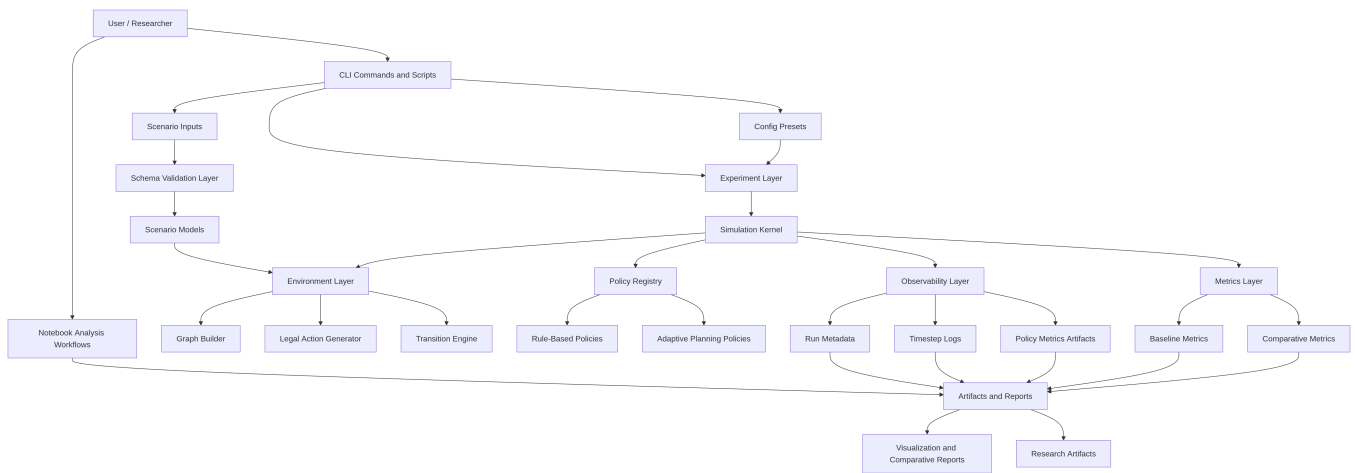


Figure 4.1: Architecture Diagram

into typed `ScenarioConfig`, `NodeConfig`, and `EdgeConfig` objects before they are used by the runtime. The environment layer is `src/environment/`. It converts validated scenarios into a `networkx.Graph`, generates legal Red and Blue actions from the current graph state, and applies controlled state transitions such as `exploit`, `patch`, `isolate`, and `block`. The agent layer is `src/agents/`. It exposes a stable policy interface through `AgentPolicy` and `PolicyRegistry`, and currently contains deterministic rule-based Red and Blue policies as well as an adaptive planning policy with optional deception and model-routing support. The simulation layer is `src/simulation/`. The main kernel executes the timestep loop, builds policy context objects, invokes policy selection, applies transitions, computes state differences, and creates structured timestep logs. The observability and metrics layer is `src/observability/` and `src/metrics/`. It serializes run metadata, timestep events, and policy metrics, validates event envelopes, reconstructs replay bundles, and computes baseline metrics such as compromise progression and containment behavior. The experimentation layer is `src/experiments/`. It runs repeated seeded simulations, policy comparison matrices, stress-test suites, and ablation packages. The visualization and reporting layer is `src/visualization/`. It generates comparative JSON/Markdown reports and SVG figures such as compromise trend, defense efficiency, and response latency. The delivery layer is `src/marbtsccli/`, `docker/`, and `notebooks/`. This gives users packaged CLI commands, container profiles, and notebook workflows for analysis.

4.3 User Interface Design

The current project does not implement a web UI or desktop GUI. Its actual user interface is command-line and artifact-driven, with notebooks used as the main analysis front end.

The primary execution interface is the packaged CLI `marbts`, which dispatches subcommands such

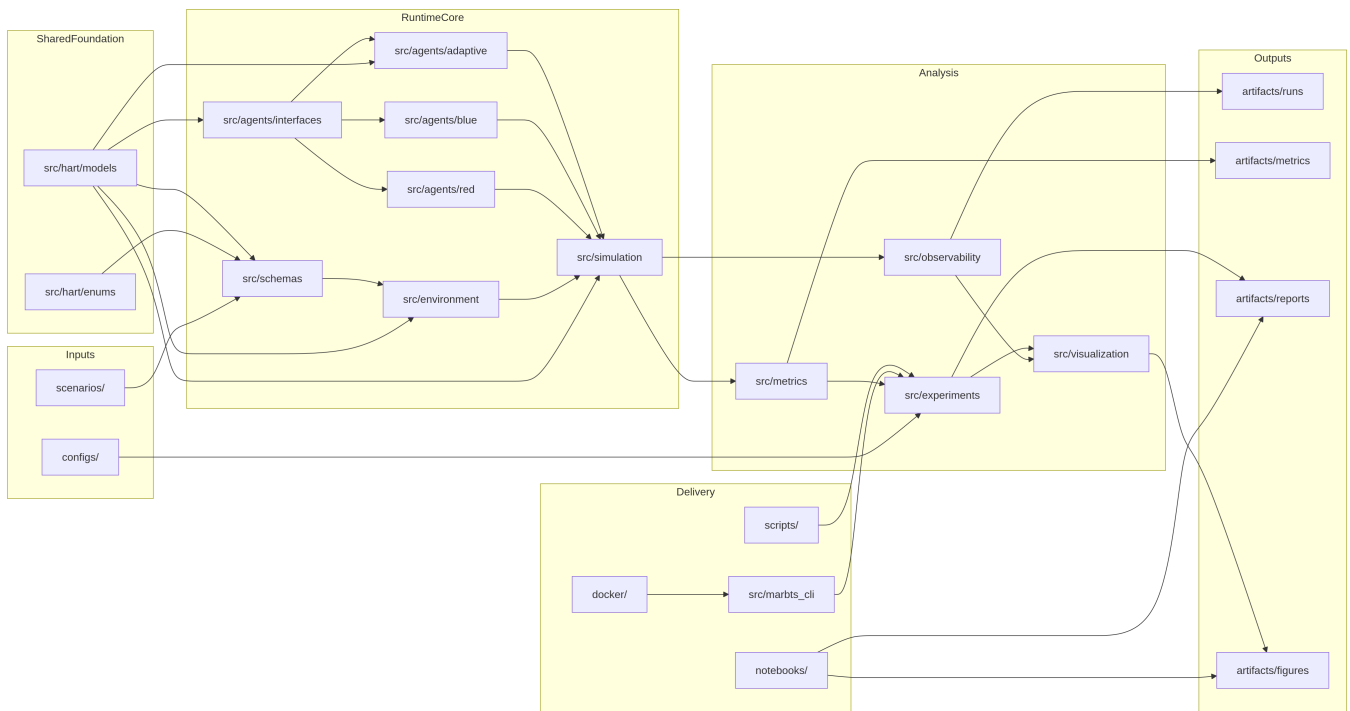


Figure 4.2: Package Level Architecture Diagram

as multi-seed-report, policy-experiment-matrix, stress-test-suite, ablation-report, container-profile, and release-validation. A second execution surface is the scripts/ folder, which provides direct script entry points for smoke runs, experiments, comparative reports, and validation tasks. The parameter design is preset-oriented. Commands accept structured inputs such as `--config`, `--scenario`, `--scenario-batch`, `--seeds`, `--horizon`, and artifact root paths, which reduces manual setup and improves reproducibility. The CLI is not interactive in the menu-driven sense; instead, it prints clear status tags and summary fields such as scenario ID, seed count, horizon, condition count, and report file path. The analysis interface is notebook-based. The notebooks `replayandcomparativewalkthrough.ipynb`, `polycymatrixwalkthrough.ipynb`, and `ablationreportwalkthrough.ipynb` allow users to inspect generated artifacts and optionally regenerate them using a `RUNGENERATORS` control flag. The output interface is file-oriented. Run artifacts are written under `artifacts/runs/`, metrics under `artifacts/metrics/`, reports under `artifacts/reports/`, and figures under `artifacts/figures/`. Containerized execution is also part of the interface design. Docker Compose profiles provide repeatable command surfaces for the same canonical experiment presets. So, the UI design of MARBTS is best described as a research workflow interface rather than a graphical application interface.

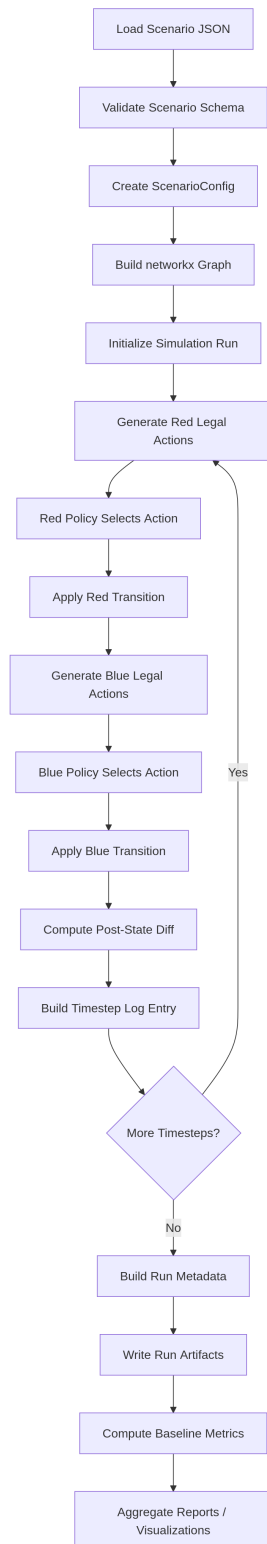


Figure 4.3: Runtime execution Flow

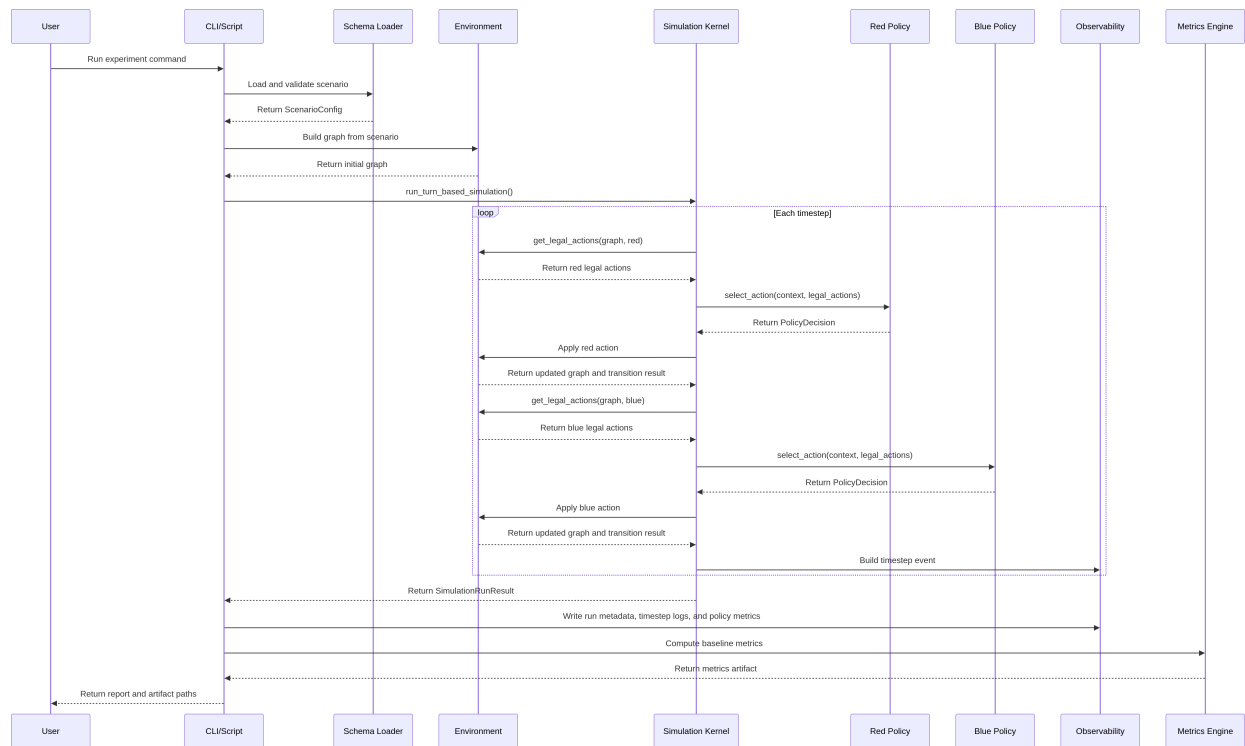


Figure 4.4: Simulation Sequence

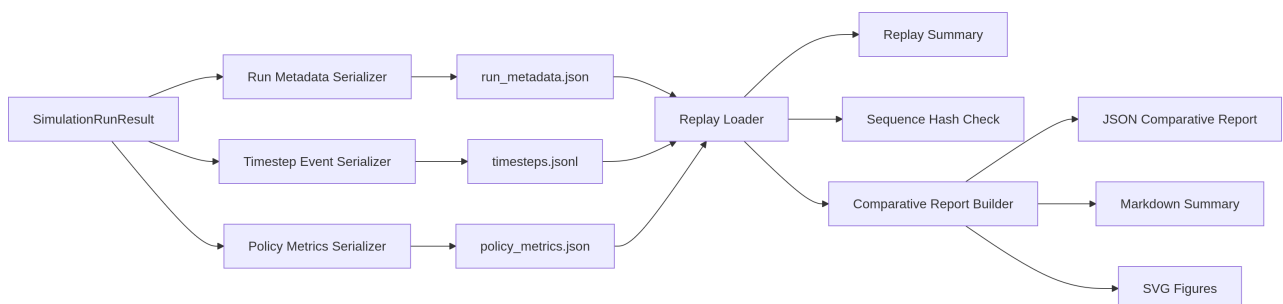


Figure 4.5: Observability and Artifact pipeline

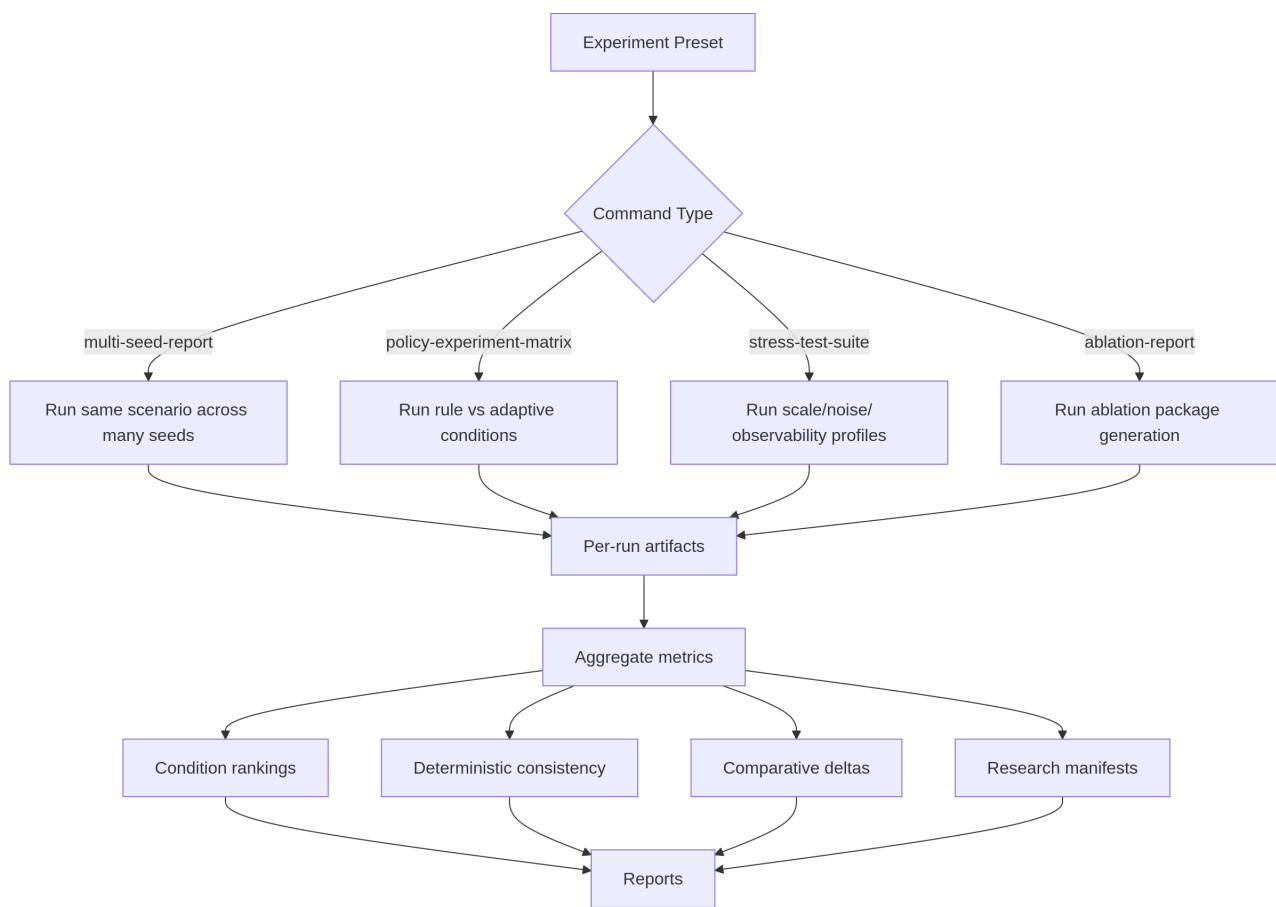


Figure 4.6: Experimnet and Reporting Workflow

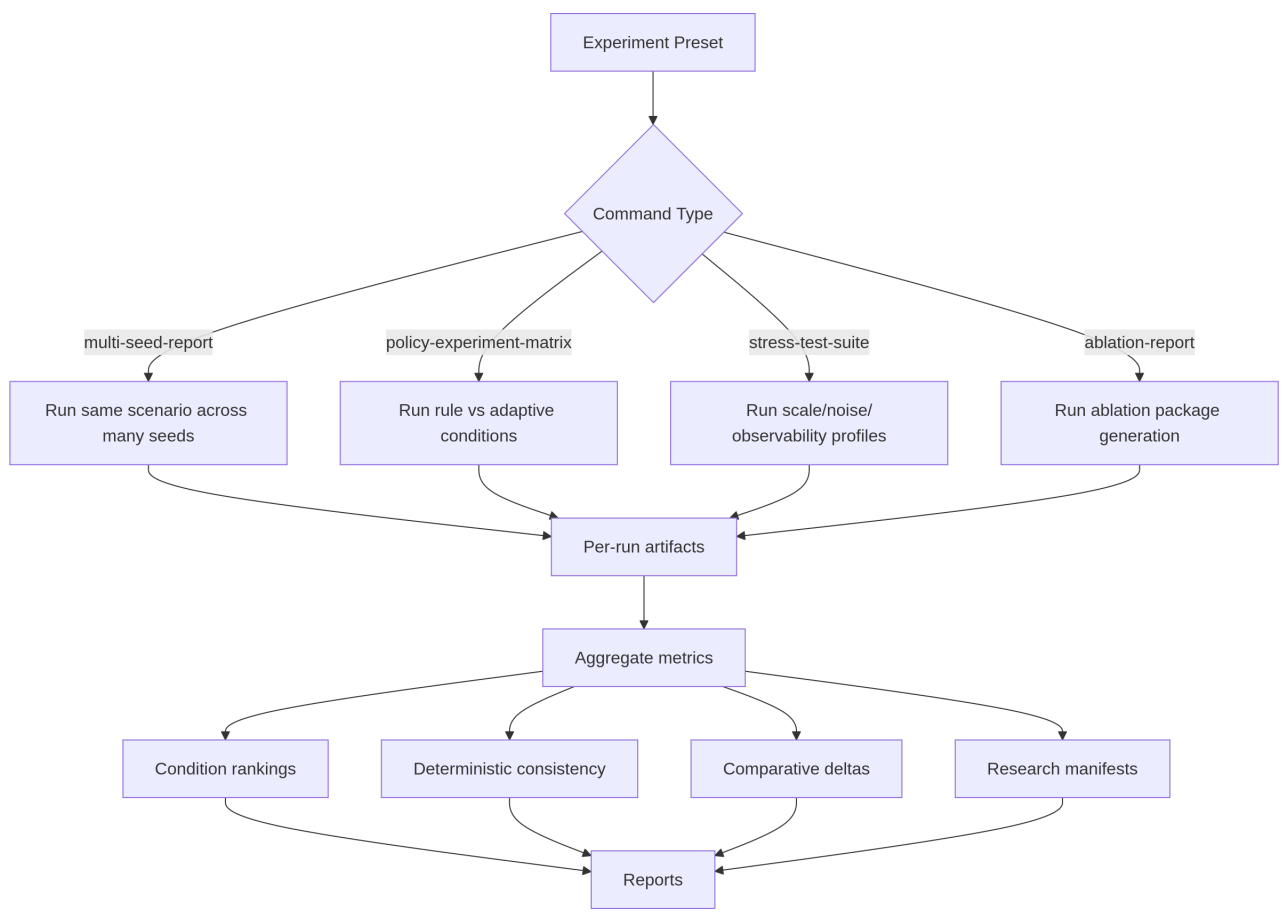


Figure 4.7: Deployment and Access Surface

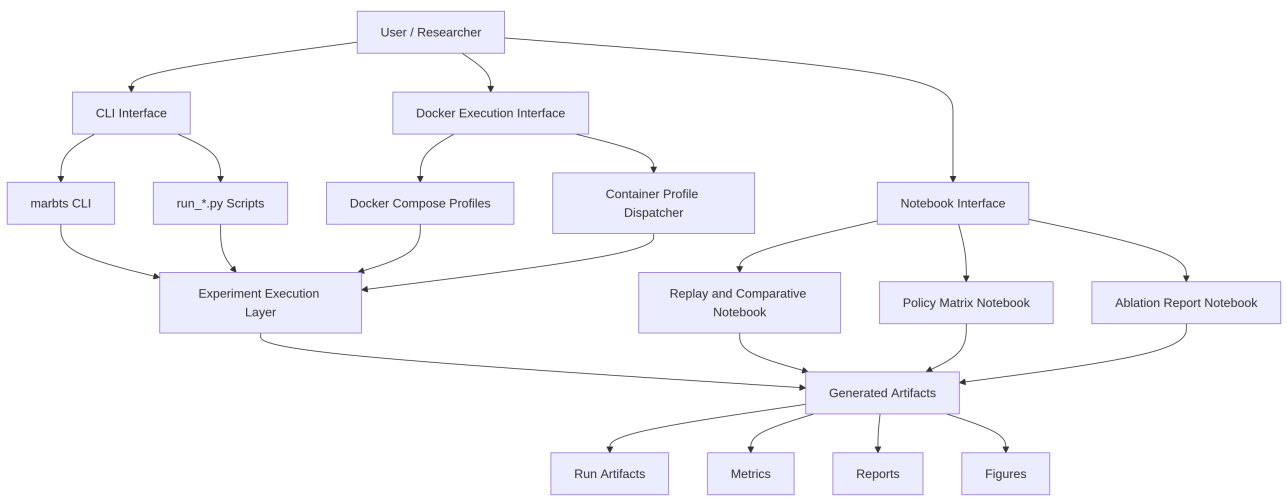


Figure 4.8: User Interaction Overview

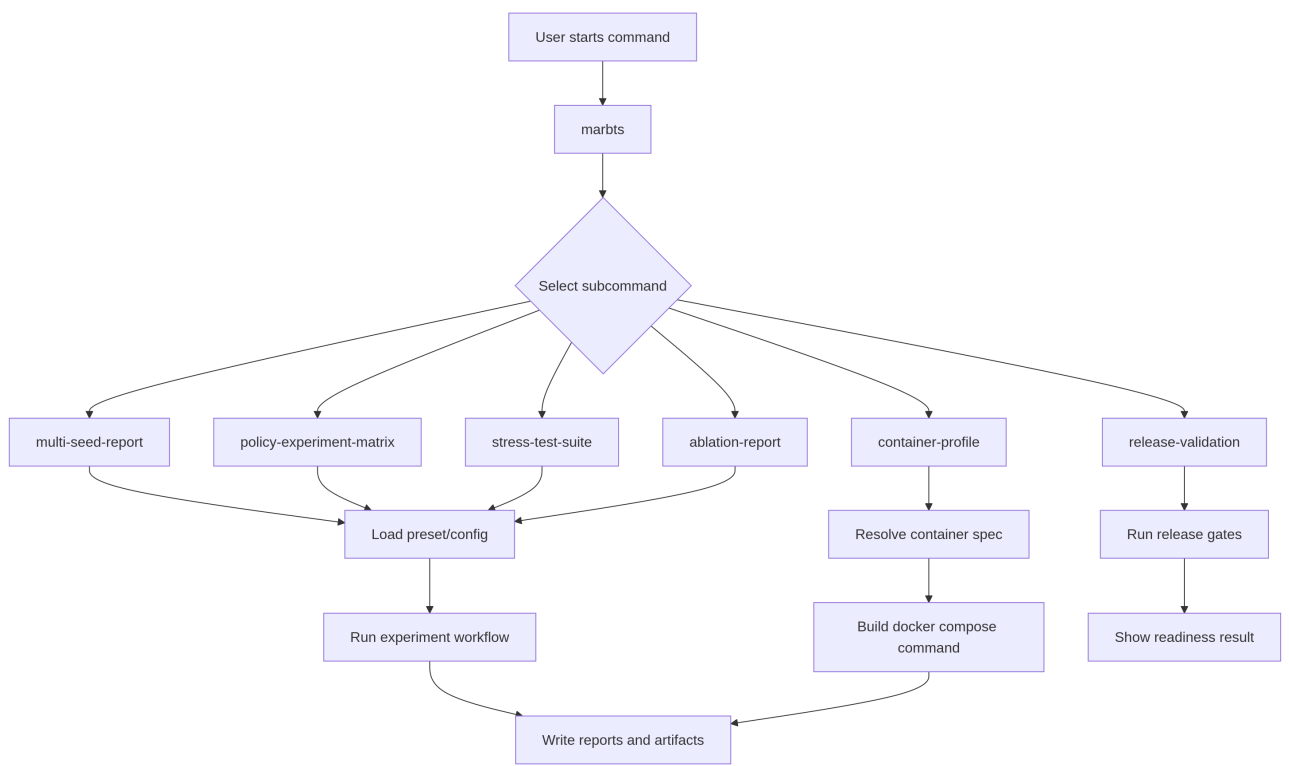


Figure 4.9: CLI Navigation Flow

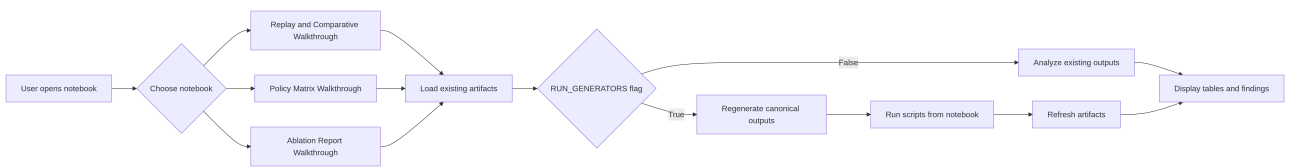


Figure 4.10: Notebook-Based Analysis UI

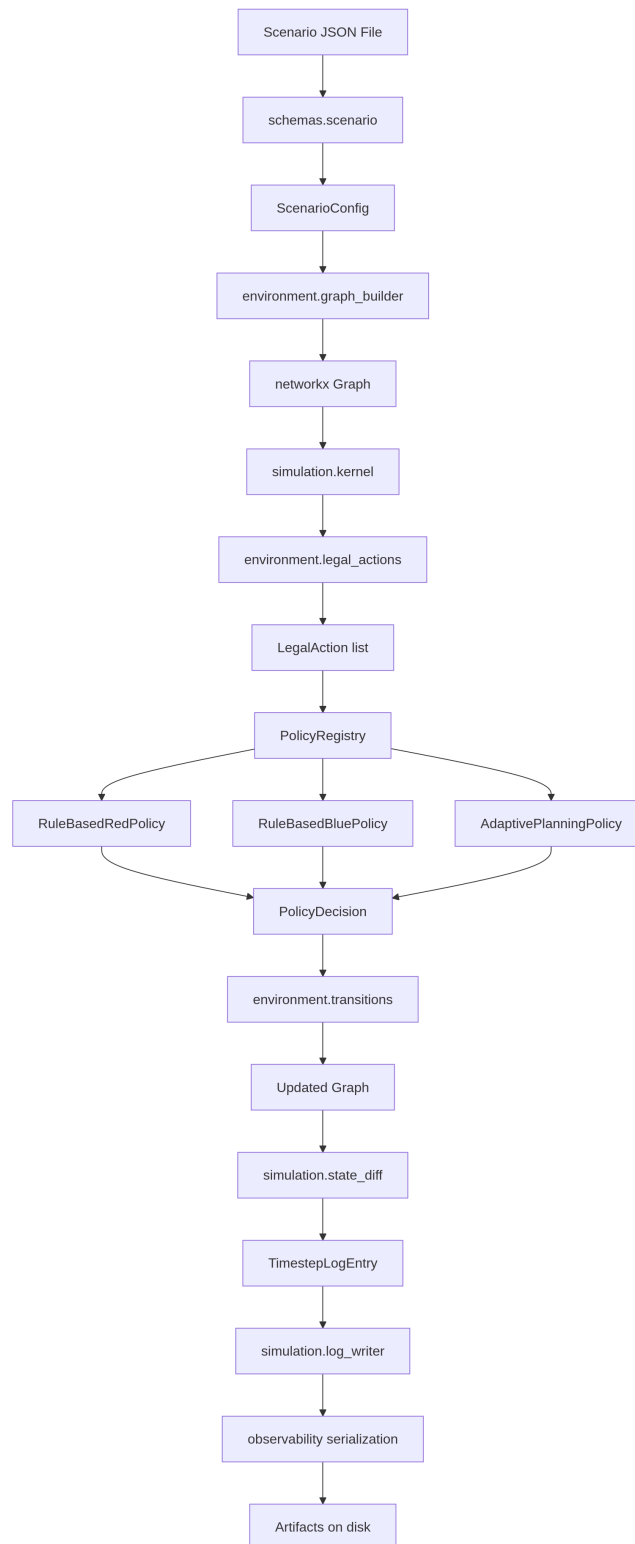


Figure 4.11: Core Module Interaction

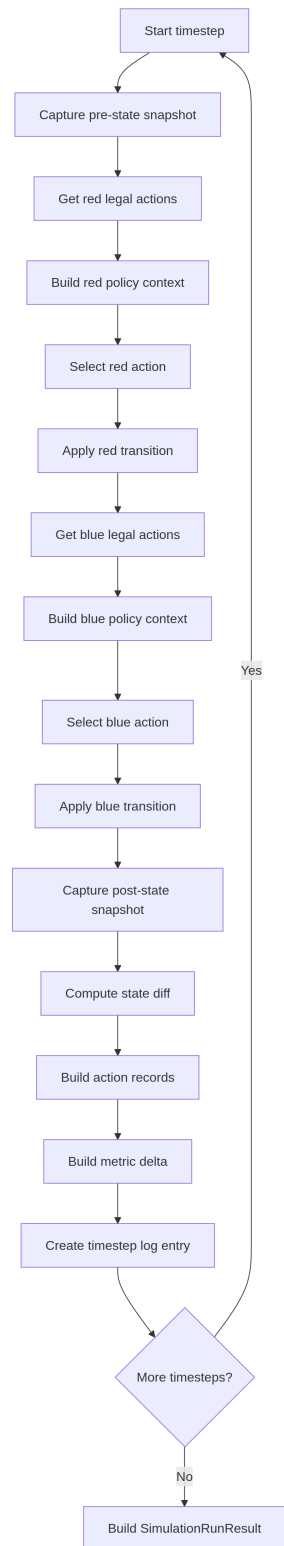


Figure 4.12: Timestamp Execution Flow

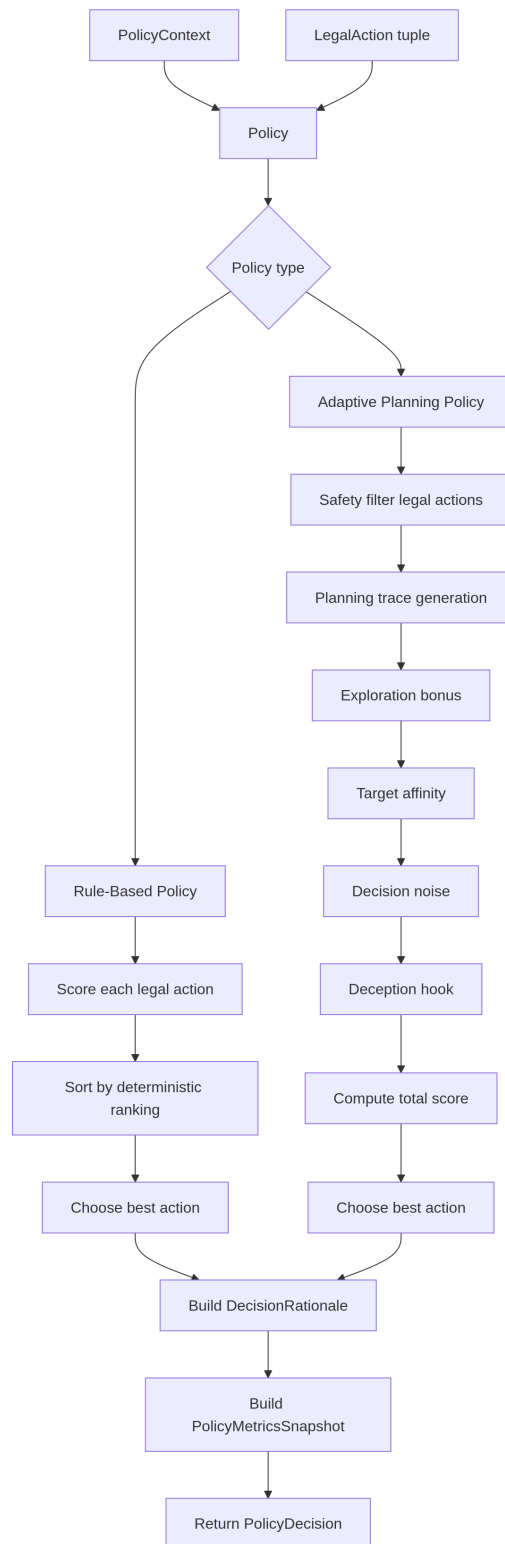


Figure 4.13: Policy Decision Internals

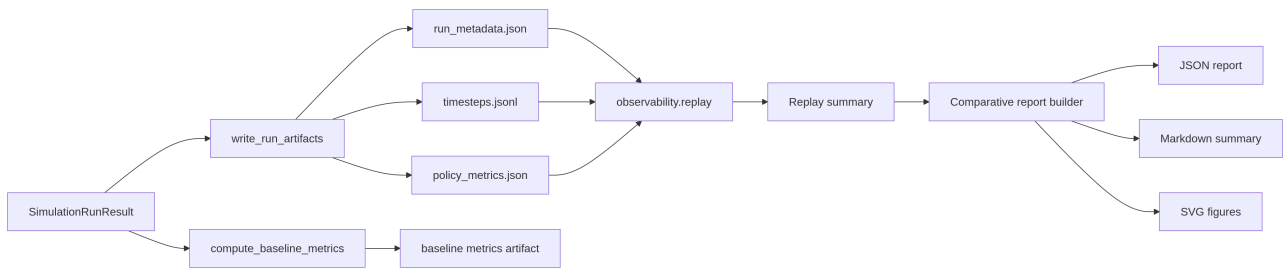


Figure 4.14: Artifact Generation Pipeline

4.4 Low Level Design

At the low level, the system is built around typed dataclasses, a networkx graph runtime, deterministic seed control, and single-responsibility modules.

Scenario ingestion begins in `src/schemas/scenario.py`, where `parsenode`, `parseedge`, `validatescenariodict`, and `loadscenariofile` enforce required fields and convert raw JSON into typed scenario objects. State representation is handled in two forms: a graph form used by the live simulation and a lightweight state form using `SimulationState` and `NodeRuntimeState`. Each node carries attributes such as `nodetype`, `services`, `vulnerabilities`, `securitylevel`, `compromisedstate`, `detectionstate`, and `isolationstate`. Action generation is implemented in `getlegalactions()`. Red can receive actions such as `scan`, `exploit`, `escalate`, and `lateralmove`, while Blue can receive `monitor`, `patch`, `isolate`, and `block`. These are generated from the current graph state, not hard-coded action sequences. The transition engine is implemented through `applyexploit()`, `applypatch()`, `applyisolate()`, and `applyblock()`. `applyexploit()` can be probabilistic when a seeded RNG is passed and uses node security level as resistance. `applypatch()` increases security, reduces compromise by one level, and removes one vulnerability. `applyisolate()` disconnects the node by removing its edges. `applyblock()` removes a specific network edge. The policy engine uses `PolicyDecision`, `DecisionRationale`, `PolicyContext`, and `PolicyMetricsSnapshot`. Rule-based policies compute weighted heuristic scores and choose the highest-ranked action deterministically. The adaptive policy computes planning traces across a horizon, target affinity, exploration bonus, decision noise, and optional deception bonuses before selecting an action. The simulation kernel `runturnbasedsimulation()` creates a seeded random generator, derives a run ID from `scenario/seed/horizon/initial state hash`, iterates over timesteps, builds actor-specific policy contexts, applies Red then Blue actions, computes pre/post snapshot references, and stores a `TimestepLogEntry` for each turn. State comparison is handled by `snapshotpayload()`, `snapshotref()`, and `computepoststatediff()`, which produce hashable state references and explicit node/edge differences between timesteps. Artifact writing is handled by `writerunartifacts()`, which emits `runmetadata.json`, `timesteps.jsonl`, and `policymetrics.json`. Provenance collection adds schema

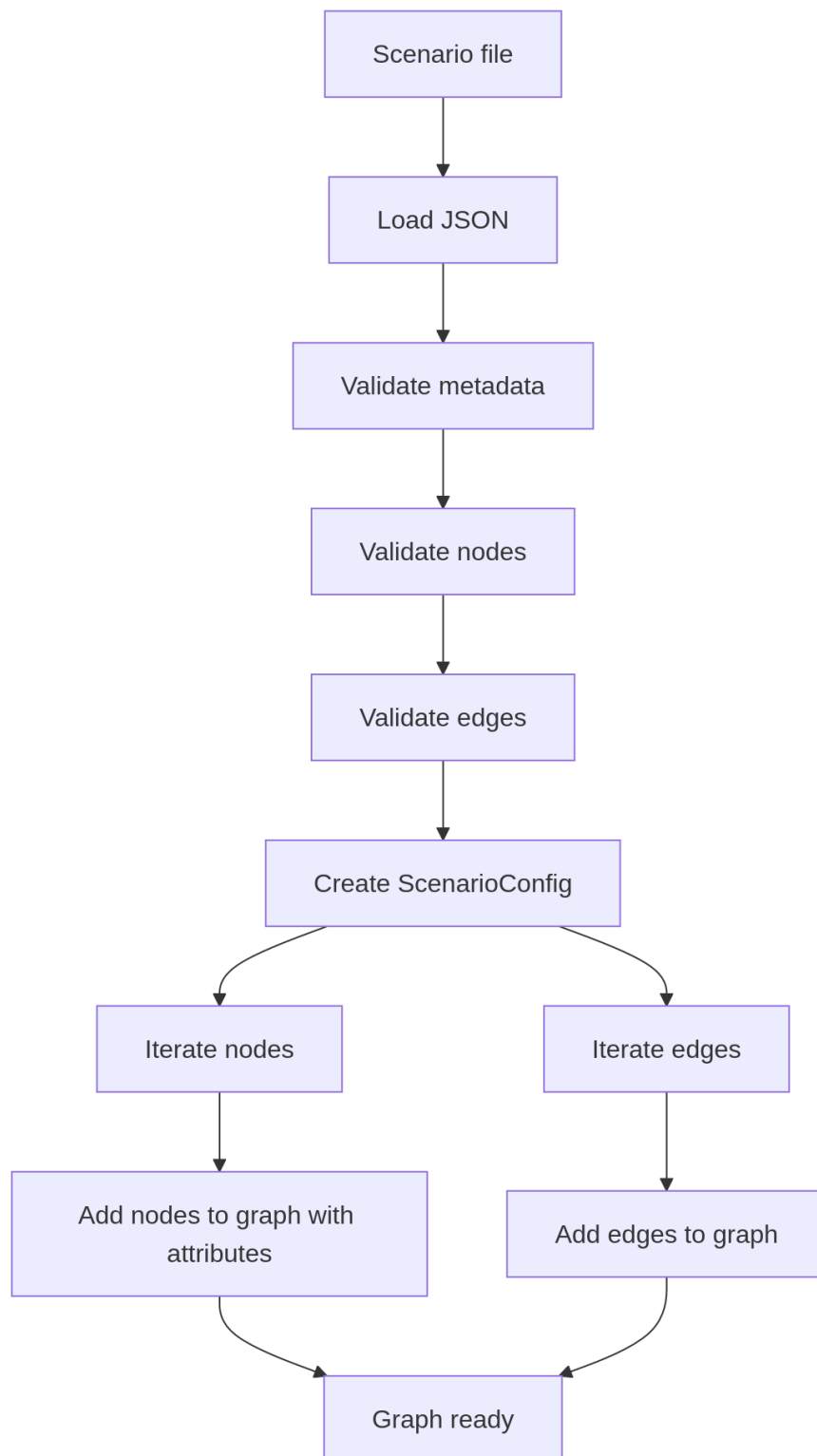


Figure 4.15: Scenario Validation and Graph Build

version, config hash, commit hash, run ID, seed, horizon, and timestamp. Replay and validation are handled by `loadreplayframes()`, `loadrunartifactbundle()`, and `validateeventpayload()`. This allows later reconstruction of action sequences and verification that logged artifacts conform to the observability contract. Experiment aggregation is implemented in modules such as `runmultiseedreport()` and `run-policyexperimentmatrix()`, which repeat seeded simulations, collect metrics, and compute aggregates like final compromise mean, containment behavior, and deterministic consistency ratio. Visualization is implemented with Matplotlib using the non-interactive Agg backend, and the reporting layer writes comparative JSON, Markdown, and SVG outputs rather than screen-based dashboards.

Chapter 5

IMPLEMENTATION

5.1 Pseudocode

The implementation of the proposed MARBTS system can be understood through the following core algorithms.

Algorithm 1: Scenario Validation and Graph Construction

Input: Scenario file in JSON format

Output: Validated scenario object and graph-based network model

1. Read the scenario file.
2. Check whether metadata, nodes, and edges are present.
3. Validate scenario identifier and version.
4. For each node:
 - a. Validate required fields such as node id, node type, services, vulnerabilities, security level, compromise state, detection state, and isolation state.
 - b. Convert the node into an internal node configuration object.
5. For each edge:
 - a. Validate source and target nodes.
 - b. Convert the edge into an internal edge configuration object.
6. Create a scenario configuration object.
7. Create an empty graph.
8. Add all validated nodes to the graph with their attributes.

9. Add all validated edges to the graph.
10. Return the scenario configuration and graph.

Algorithm 2: Turn-Based Simulation Execution

Input: Initial graph, seed, horizon, scenario id

Output: Simulation run result with logs and final graph

1. Initialize a seeded random generator.
2. Register default Red and Blue policies.
3. Copy the initial graph into the current graph.
4. Generate a unique run identifier.
5. For each timestep from 0 to horizon - 1:
 - a. Capture the pre-state snapshot.
 - b. Generate legal Red actions from the current graph.
 - c. Build Red policy context.
 - d. Select Red action using the Red policy.
 - e. Apply Red action to the graph.
 - f. Generate legal Blue actions from the updated graph.
 - g. Build Blue policy context.
 - h. Select Blue action using the Blue policy.
 - i. Apply Blue action to the graph.
 - j. Capture the post-state snapshot.
 - k. Compute state difference between pre-state and post-state.
 - l. Create timestep log entry with action records and metric delta.
6. Build run metadata.
7. Return the final simulation result.

Algorithm 3: Policy Action Selection

Input: Policy context and legal actions

Output: Selected action, rationale, and policy metrics

1. Receive the current policy context and legal actions.
2. If the policy is rule-based:

- a. Score each legal action using deterministic heuristics.
 - b. Rank actions according to score and tie-breaking rules.
 - c. Select the highest-ranked action.
3. If the policy is adaptive:
 - a. Filter actions for safety and validity.
 - b. For each safe action:
 - i. Estimate planning utility over a fixed horizon.
 - ii. Compute exploration bonus.
 - iii. Compute target preference from node security level.
 - iv. Add optional decision noise and deception bonus.
 - c. Rank all candidate actions.
 - d. Select the highest-ranked action.
4. Build rationale describing why the action was selected.
5. Update policy metrics.
6. Return the final policy decision.

Algorithm 4: Artifact and Report Generation

Input: Simulation run result or batch of run results

Output: Logs, metrics, reports, and visual artifacts

1. Store run metadata.
2. Store timestep logs for all simulation steps.
3. Store policy metrics and action sequence summary.
4. Compute baseline metrics such as:
 - a. final compromised nodes,
 - b. containment actions,
 - c. first containment timestep,
 - d. deterministic consistency.
5. For multi-seed execution:
 - a. Repeat simulation for each seed.
 - b. Aggregate results using mean, standard deviation, minimum, and maximum values.
6. For policy matrix execution:

- a. Run baseline and adaptive policy combinations.
 - b. Compare each condition with the baseline.
 - c. Rank conditions by defensive performance.
7. Generate final report files and visual summaries.

5.2 Explanation

The implementation of MARBTS is modular and follows the architecture defined for the project. The complete system is divided into scenario handling, environment modeling, agent decision-making, simulation control, observability, metrics, and experiment execution. This modular structure makes the project easier to test, extend, and analyze.

Scenario and Input Handling

The first stage of implementation is scenario loading and validation. The system reads scenario files defined in JSON format and checks whether the metadata, node information, and edge definitions are correct. Each node contains attributes such as its type, services, vulnerabilities, security level, compromise state, detection state, and isolation state. After validation, the scenario is converted into an internal configuration object. This validated input is then used to build a graph-based network model, where nodes represent devices or hosts and edges represent communication links.

Environment Modeling

The environment is implemented as a graph-based cyber defense model. Once the graph is created, the system derives the set of legal actions that can be taken by the Red and Blue agents at any timestep. Red-side actions include scanning, exploitation, privilege escalation, and lateral movement. Blue-side actions include monitoring, patching, blocking network paths, and isolating compromised nodes. Each action is applied using controlled transition rules so that the graph changes only through valid state updates. This design ensures that the simulation remains abstract, safe, and reproducible.

Agent Implementation

The project implements both rule-based and adaptive agents. The rule-based agents use deterministic heuristic scoring to choose actions. Each available action is assigned a score based on factors such as expected gain, containment urgency, resilience impact, or threat pressure. The action with the best score is selected.

The adaptive agent extends this logic by introducing short-horizon planning. Instead of only looking at the immediate action, it estimates the possible utility of an action over multiple future steps. It also includes optional features such as reduced observability, deception hooks, and bounded decision noise. For every selected action, the system stores an explanation containing the rationale, expected effect, confidence, and score breakdown. This makes the agent behavior explainable and suitable for research analysis.

Simulation Kernel

The simulation kernel is the core of the implementation. It controls the timestep-by-timestep interaction between the Red and Blue agents. At each timestep, the kernel records the current graph state, requests legal actions, calls the policy layer to select actions, applies state transitions, and then computes the difference between the old and new system states. The result is stored as a structured timestep log. The simulation continues until the configured horizon is reached. A seeded random number generator is used wherever stochastic behavior is needed, which guarantees reproducibility when the same seed is used again.

Observability and Logging

A major part of the implementation is the observability layer. After each simulation run, the system generates structured artifacts including run metadata, timestep logs, and policy metrics. The metadata records the run identifier, seed, scenario, horizon, timestamp, and provenance details. The timestep logs store both Red and Blue actions, their rationale, the resulting state differences, and metric changes. These artifacts make it possible to replay the execution, verify consistency, and compare different experimental runs.

Metrics and Evaluation

The metrics layer computes the quantitative outcomes of each simulation. Examples include the number of compromised nodes at the end of the run, the maximum compromise level reached during the run, the number of containment actions performed by Blue, and the timestep at which containment first occurred. For multi-seed experiments, the system aggregates these results across several runs to calculate means, standard deviations, and consistency ratios. These metrics provide a clear basis for comparing different policy types and different experiment conditions.

Experiment Workflows

The implementation also includes higher-level experiment workflows. In the multi-seed workflow, the same scenario is executed across multiple seeds to study repeatability and statistical stability. In the policy matrix workflow, rule-based and adaptive Red and Blue policies are compared under baseline and ablation settings. In the stress-test workflow, multiple scenarios are executed together to analyze robustness under scale and observability constraints. In the ablation workflow, the system produces research-oriented report packages showing how planning, observability, and deception features affect performance.

Output and Deliverables

The final implementation produces several types of outputs. These include raw run artifacts, metric summaries, comparative reports, markdown summaries, and visualization files. The outputs are organized into separate artifact folders so that results from different runs and experiments can be stored and reused. Command-line interfaces, configuration presets, and notebook workflows are also included to make execution and analysis easier for researchers and student users.

Summary

Overall, the implementation of MARBTS translates the project design into a working research platform. The codebase integrates validated scenario input, graph-based simulation, policy-driven decision-making, reproducible execution, structured logging, and experiment-level reporting. This combination allows the system to function not only as a simulator, but also as a complete research environment for studying autonomous cyber defense strategies.

Chapter 6

RESULTS

6.1 Result Snapshots

This chapter presents the experimental results obtained from the implemented MARBTS platform. The results were generated by executing the baseline multi-seed workflow, the policy experiment matrix, the comparative report workflow, the stress-test suite, and the ablation report workflow.

Snapshot 1: Rule-Based Baseline Run



Figure 6.1: Result snapshot of the Rule Red vs Rule Blue baseline execution.

Figure 6.1 shows the baseline execution for the rule-baseline scenario using seed 20260423 and horizon 2. In this run, the Red agent selected only scan actions, while the Blue agent selected only monitor actions. Since both selected actions were informational and caused no state mutation, the

graph remained unchanged throughout the run. As a result, the number of compromised nodes remained 0 at every timestep, and no containment action was triggered. This snapshot demonstrates the stable and fully deterministic behavior of the rule-based baseline configuration.

Snapshot 2: Adaptive Red vs Rule Blue Run



Figure 6.2: Result snapshot of the Adaptive Red vs Rule Blue execution.

Figure 6.2 presents the execution of the Adaptive Red vs Rule Blue condition for the same seed and horizon. In this case, the adaptive Red policy selected exploit on node iot-1 at both timesteps. During the first timestep, the node state changed from none to user, and during the second timestep it changed from user to privileged. In response, the Blue agent selected block actions and removed the edges api-1 to db-1 and api-1 to iot-1. This snapshot shows that the adaptive planner was able to generate an actual compromise progression, while the rule-based Blue defender reacted through containment-oriented edge blocking.

Snapshot 3: Comparative Result Output

Figure 6.3 represents the comparative report generated between the rule-based baseline run and the adaptive Red run. The report confirms that the baseline run ended with 0 compromised nodes, whereas the adaptive Red run ended with 1 compromised node. It also shows that Blue performed



Figure 6.3: Snapshot of the comparative report generated between the baseline run and the adaptive Red run.

2 containment actions in the adaptive case and none in the baseline case. This snapshot is useful because it presents both state progression and response behavior in a single consolidated output.

6.2 Comparison

This section compares the results obtained from the main experimental workflows. The first level of comparison is the multi-seed baseline summary. The second level compares major policy combinations from the policy experiment matrix. The third level summarizes the ablation ranking behavior produced by the research-oriented reporting workflow.

Multi-Seed Baseline Comparison

Table 6.1: Multi-seed baseline summary for the rule-baseline scenario.

Scenario	Seeds	Horizon	Final Compromised Mean	Blue Containment Mean	Deterministic Consistency
Rule-baseline	3	8	0.0	0.0	1.0

Table 6.1 shows that the baseline multi-seed experiment remained perfectly stable across all three seeds. The mean number of compromised nodes was 0.0, the mean number of containment actions was also 0.0, and the deterministic consistency ratio was 1.0. This indicates that the baseline workflow is fully reproducible for the tested seed set and that the selected rule-based policies produce identical action sequences under the same scenario structure.

Policy Condition Comparison

Table 6.2: Comparison of selected policy combinations from the policy experiment matrix.

Condition	Final Compromised Mean	Blue Containment Mean	Deterministic Consistency
Rule Red vs Rule Blue	0.0	0.0	1.0
Adaptive Red vs Rule Blue	1.0	2.0	1.0
Rule Red vs Adaptive Blue	0.0	0.0	1.0
Adaptive Red vs Adaptive Blue	1.0	0.0	1.0

Table 6.2 highlights the behavioral differences between the four core policy configurations. The Rule Red vs Rule Blue condition produced no compromise and no containment actions, showing a conservative and non-escalating baseline. The Adaptive Red vs Rule Blue condition resulted in a mean of 1.0 compromised node and 2.0 Blue containment actions, indicating that the adaptive Red strategy introduced enough offensive pressure to force defensive response. The Rule Red vs Adaptive Blue condition again produced 0.0 final compromised nodes, which suggests that the adaptive Blue agent was at least as effective as the rule-based defender in the tested baseline scenario. The Adaptive Red vs Adaptive Blue condition still ended with 1.0 compromised node, but with no containment action mean, showing a different action-selection pattern from the rule-based Blue defender.

Ablation Ranking Summary

The ablation report ranked the conditions according to final compromise outcome. The highest-ranked group consisted of Rule Red vs Adaptive Blue and its adaptive Blue ablation variants, all of which produced a final compromised mean of 0. The standard Rule Red vs Rule Blue baseline also remained in this zero-compromise group. In contrast, the conditions where Red used adaptive planning produced a final compromised mean of 1, regardless of whether deception hooks or reduced observability were enabled. This indicates that, for the current baseline scenario and horizon, the main performance difference was driven more strongly by whether the Red side used adaptive planning than by the ablation flags themselves.

6.3 Performance Analysis

This section analyzes the generated figures and stress-test outputs to understand how compromise progression, defense efficiency, and scenario complexity affect overall system behavior.

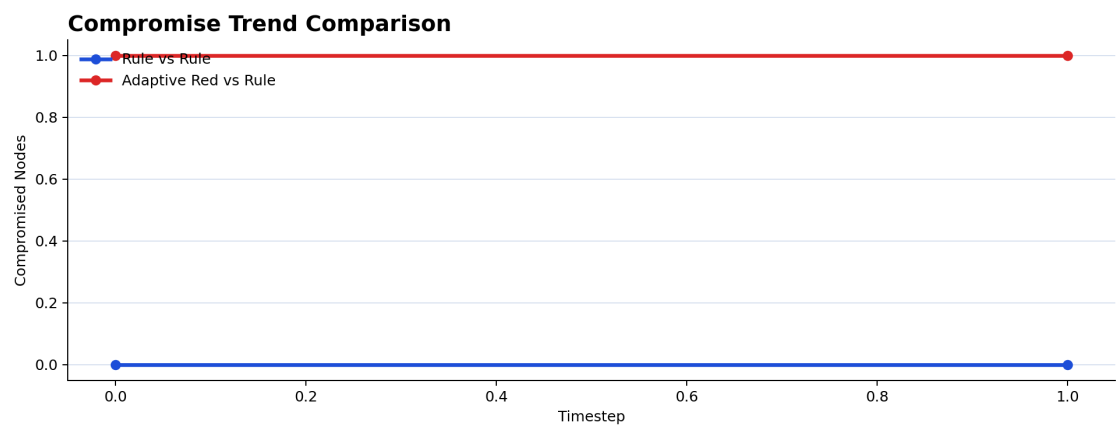


Figure 6.4: Compromise trend comparison between the baseline run and the Adaptive Red vs Rule Blue run.

Compromise Trend Analysis

Figure 6.4 shows the difference in compromise progression between the two selected runs. In the rule-based baseline, the number of compromised nodes remained at 0 for all timesteps. In contrast, the adaptive Red run reached 1 compromised node immediately and maintained that compromised state until the end of execution. This figure clearly illustrates the impact of the adaptive Red agent in generating an actual offensive progression that was absent in the baseline case.

Defense Efficiency Analysis

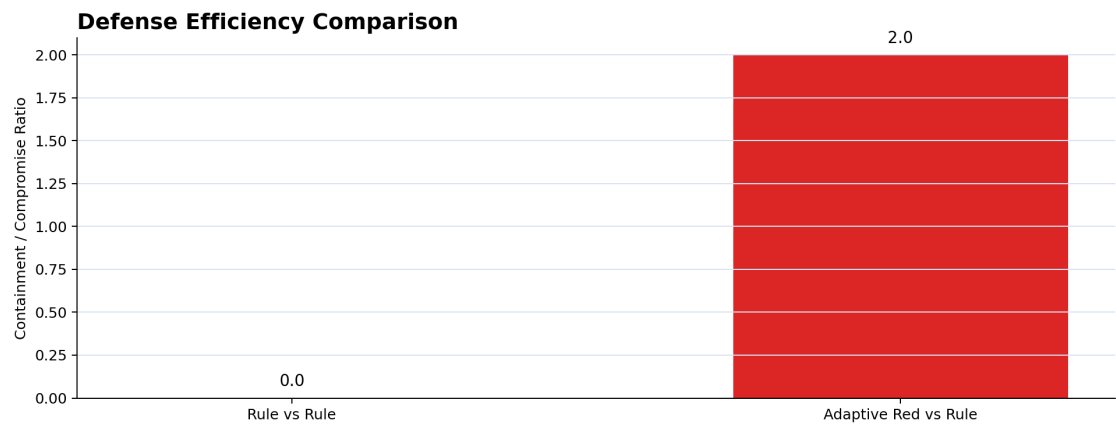


Figure 6.5: Defense efficiency comparison using containment-to-compromise ratio.

Figure 6.5 compares defense efficiency using the containment-to-compromise ratio. The baseline run produced a ratio of 0.0 because no compromise occurred and no containment action was required. The adaptive Red run produced a ratio of 2.0, meaning that the defender executed two containment actions for one compromised node. This indicates that the rule-based Blue agent responded actively once offensive activity was present, although the response did not fully prevent

compromise progression.

Response Latency Analysis

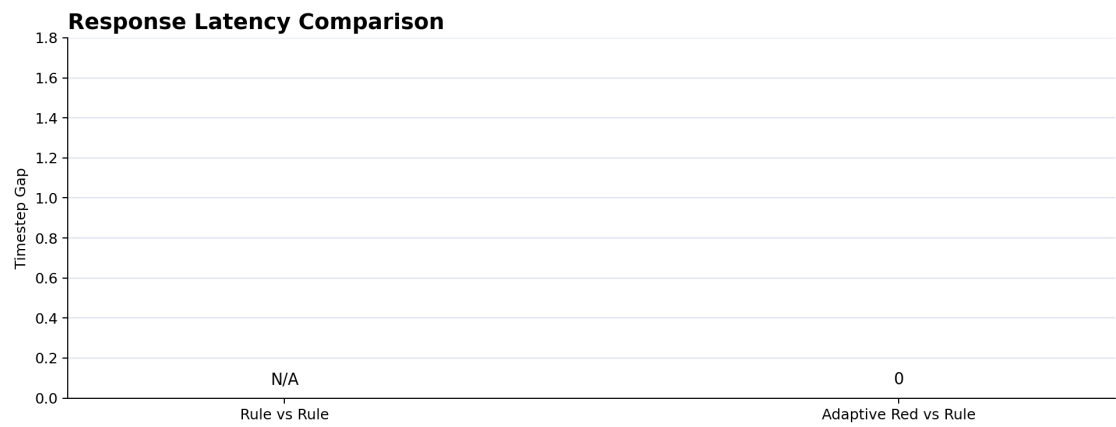


Figure 6.6: Response latency comparison between the baseline run and the adaptive Red run.

Figure 6.6 compares response latency, defined as the gap between the first compromise and the first containment action. In the baseline case, the value is not applicable because no compromise occurred. In the adaptive Red run, the latency was 0, which means that Blue responded during the very first timestep in which compromise appeared. Although the response was immediate, the final compromised node count still remained higher than the baseline, showing that fast response alone does not guarantee full prevention.

Stress-Test Profile Analysis

Table 6.3: Stress-test profile summary generated from the stress-test suite.

Profile	Scenario Count	Seed Count	Baseline Final Compromised Mean	Combined Observability Penalty	Rank
Scale Scenarios Stress	3	2	0.667	Not Applicable	1
Noise and Observability Constraints	2	2	1.0	0.0	2

Table 6.3 summarizes the two profiles generated by the stress-test suite. The Scale Scenarios Stress profile ranked first, with a baseline final compromised mean of 0.667. The Noise and Observability Constraints profile ranked second, with a mean of 1.0. The observability penalty summary for the second profile produced a combined penalty mean of 0.0 over four samples, indicating that reduced observability did not worsen the final compromise metric in this specific experiment configuration. This suggests that, within the tested scope, scenario complexity had a more visible effect on the baseline outcome than the observability constraint proxy.

Overall Interpretation

From the collected results, three main observations can be made. First, the baseline workflows are highly reproducible, as shown by the deterministic consistency ratio of 1.0. Second, adaptive Red policies generate stronger offensive behavior than rule-based Red policies in the tested baseline scenario, which directly increases compromise-related metrics and triggers more Blue containment actions. Third, adaptive Blue policies performed competitively in the current baseline setting, particularly in the zero-compromise group of the policy matrix and ablation rankings. Overall, the implemented MARBTS platform successfully produced structured, explainable, and reproducible outputs that can be used for both academic reporting and further research analysis.

Chapter 7

CONCLUSION & FUTURE WORK

7.1 Conclusion

The MARBTS project successfully achieved its main objective of developing a safe, modular, explainable, and reproducible Red vs Blue autonomous cyber defense simulation platform. The system models attacker and defender interactions within a synthetic graph-based network and provides a controlled environment for studying compromise progression, defensive response, and policy behavior without relying on real-world targets or unsafe offensive tooling. By keeping the entire framework sandboxed and abstract, the project demonstrates that meaningful cybersecurity experimentation can be performed in a responsible and research-oriented manner.

A major contribution of this work is the integration of multiple capabilities into a single coherent platform. The project includes validated scenario handling, graph-based environment construction, legal action generation, rule-based and adaptive policy support, turn-based simulation, structured observability, baseline metrics, comparative reporting, stress-test workflows, and ablation-oriented research packaging. The implementation also emphasizes clarity and accountability by attaching rationale and decision traces to agent actions, while reproducibility is ensured through seeded execution, structured artifacts, provenance capture, and deterministic validation workflows. These features make the platform useful not only as a simulator, but also as a complete experimentation framework for analysis and benchmarking.

The results obtained from the implemented workflows show that the system behaves consistently across repeated runs and is capable of capturing meaningful differences between policy types. Baseline runs remained fully reproducible, while adaptive-policy conditions produced observable changes in compromise behavior, containment response, and comparative metrics. This confirms that the project is effective as a foundation for evaluating autonomous cyber defense strategies. Overall,

MARBTS can be considered a successful student project because it combines sound software design, practical implementation, research utility, and future extensibility in a technically relevant problem domain.

7.2 Future work

Although the current implementation is functionally strong, there are several directions in which the project can be extended. One important area is the improvement of adaptive autonomy. At present, the project includes adaptive planning-based behavior with bounded decision logic, but future versions can incorporate richer learning-driven strategies such as reinforcement learning, stronger planning methods, or carefully controlled large language model reasoning. This would make the system more suitable for advanced policy evaluation while preserving the same sandboxed and explainable design principles.

Another major area for future work is expansion of the simulation environment itself. The scenario library can be extended to include larger and more diverse enterprise-style topologies, richer node types, more detailed defensive controls, service availability constraints, and more realistic observability limitations. Additional deception mechanisms, uncertainty models, and robustness-oriented perturbation workflows can also be integrated to support broader cybersecurity research questions. These extensions would allow the platform to represent more challenging attacker-defender dynamics while still remaining safe for experimentation.

Future enhancement is also possible at the usability and product level. The current system already provides command-line workflows, Docker support, notebooks, and release validation, but future versions can introduce a more user-friendly dashboard or web-based interface for running experiments and inspecting results. Visualization can be improved with more interactive reports, richer replay tools, and clearer comparative analytics. Documentation, installation flow, and collaborator onboarding can also be simplified further so that the platform becomes easier to adopt for academic projects, research groups, and educational cybersecurity laboratories.

In summary, the future work of MARBTS lies in making the platform more intelligent, more expressive, and more accessible, while preserving its core strengths of safety, modularity, explainability, and reproducibility. Because the present implementation already establishes a strong architectural and experimental foundation, these future enhancements can be added with minimal disruption and can significantly increase the long-term value of the project.

References

- [1] D. Miller, R. Alford, A. Applebaum, H. Foster, C. Little, and B. Strom, “Automated Adversary Emulation: A Case for Planning and Acting with Unknowns,” MITRE, McLean, VA, USA, Jun. 2018. [Online]. Available: <https://www.mitre.org/news-insights/publication/automated-adversary-emulation-case-planning-and-acting-unknowns>
- [2] MITRE, “CALDERA,” MITRE, Mar. 2025. [Online]. Available: <https://www.mitre.org/our-impact/intellectual-property/caldera>
- [3] V. S. Devi Priya and S. Sibi Chakkaravarthy, “Containerized Cloud-Based Honeypot Deception for Tracking Attackers,” *Scientific Reports*, vol. 13, Art. no. 1437, Jan. 2023.
- [4] S. Wang, Y. Li, and F. Chen, “Optimizing Blue Team Strategies with Reinforcement Learning for Enhanced Ransomware Defense Simulations,” preprint, Aug. 2024. [Online]. Available: <https://doi.org/10.22541/au.172356133.30648910/v1>
- [5] A. Javadpour, F. Ja’fari, T. Taleb, M. Shojafar, and C. Benzaïd, “A Comprehensive Survey on Cyber Deception Techniques to Improve Honeypot Performance,” *Computers & Security*, vol. 140, Art. no. 103792, 2024.
- [6] D. S. Morozov, A. A. Yefimenko, T. M. Nikitchuk, R. O. Kolomiets, and S. O. Semerikov, “The Sweet Taste of IoT Deception: An Adaptive Honeypot Framework for Design and Evaluation,” *Journal of Edge Computing*, vol. 3, no. 2, pp. 207–223, Nov. 2024.
- [7] C. R. Landolt, C. Würsch, R. Meier, A. Mermoud, and J. Jang-Jaccard, “Multi-Agent Reinforcement Learning in Cybersecurity: From Fundamentals to Applications,” *arXiv preprint arXiv:2505.19837*, May 2025.
- [8] Q. Xie, E. Koh, X. Cadet, and P. Chin, “Nash Q-Network for Multi-Agent Cybersecurity Simulation,” *arXiv preprint arXiv:2509.00678*, Aug. 2025.

- [9] J. Soulé, J.-P. Jamont, M. Occhetto, P. Théron, and L.-M. Traonouez, “Towards a Multi-Agent Simulation of Cyber-attackers and Cyber-defenders Battles,” *arXiv preprint arXiv:2506.04849*, Jun. 2025.